Smart Contract
Security Audit Report

Table Of Contents

1 Executive Summary

2 Audit Methodology

3 Project Overview

3.1 Project Introduction

3.2 Vulnerability Information

4 Code Overview

4.1 Contracts Description

4.2 Visibility Description

4.3 Vulnerability Summary

5 Audit Result

6 Statement

1 Executive Summary

On 2026.01.26, the SlowMist security team received the Paimon team's security audit application for Paimon(PPT) BSC Contracts, developed the audit plan according to the agreement of both parties and the characteristics of the project, and finally issued the security audit report.

The SlowMist security team adopts the strategy of "white box lead, black, grey box assists" to conduct a complete security test on the project in the way closest to the real attack.

The test method information:

Test method	Description
Black box testing	Conduct security tests from an attacker's perspective externally.
Grey box testing	Conduct security testing on code modules through the scripting tool, observing the internal running status, mining weaknesses.
White box testing	Based on the open source code, non-open source code, to detect whether there are vulnerabilities in programs such as nodes, SDK, etc.

The vulnerability severity level information:

Level	Description
Critical	Critical severity vulnerabilities will have a significant impact on the security of the DeFi project, and it is strongly recommended to fix the critical vulnerabilities.
High	High severity vulnerabilities will affect the normal operation of the DeFi project. It is strongly recommended to fix high-risk vulnerabilities.
Medium	Medium severity vulnerability will affect the operation of the DeFi project. It is recommended to fix medium-risk vulnerabilities.
Low	Low severity vulnerabilities may affect the operation of the DeFi project in certain scenarios. It is suggested that the project team should evaluate and consider whether these vulnerabilities need to be fixed.
Weakness	There are safety risks theoretically, but it is extremely difficult to reproduce in engineering.
Suggestion	There are better practices for coding or architecture.

2 Audit Methodology

The security audit process of SlowMist security team for smart contract includes two steps:

- Smart contract codes are scanned/tested for commonly known and more specific vulnerabilities using automated analysis tools.
- Manual audit of the codes for security issues. The contracts are manually analyzed to look for any potential problems.

Following is the list of commonly known vulnerabilities that was considered during the audit of the smart contract:

Serial Number	Audit Class	Audit Subclass
1	Overflow Audit	-
2	Reentrancy Attack Audit	-
3	Replay Attack Audit	-
4	Flashloan Attack Audit	-
5	Race Conditions Audit	Reordering Attack Audit
6	Permission Vulnerability Audit	Access Control Audit
		Excessive Authority Audit
7	Security Design Audit	External Module Safe Use Audit
		Compiler Version Security Audit
		Hard-coded Address Security Audit
		Fallback Function Safe Use Audit
		Show Coding Security Audit
		Function Return Value Security Audit
		External Call Function Security Audit

Serial Number	Audit Class	Audit Subclass
7	Security Design Audit	Block data Dependence Security Audit
		tx.origin Authentication Security Audit
8	Denial of Service Audit	-
9	Gas Optimization Audit	-
10	Design Logic Audit	-
11	Variable Coverage Vulnerability Audit	-
12	"False Top-up" Vulnerability Audit	-
13	Scoping and Declarations Audit	-
14	Malicious Event Log Audit	-
15	Arithmetic Accuracy Deviation Audit	-
16	Uninitialized Storage Pointer Audit	-

3 Project Overview

3.1 Project Introduction

This is the Paimon asset management vault system based on the ERC4626 standard. It allows users to deposit underlying assets (such as USDT) to earn PPT shares. The protocol manages multi-tiered investment strategies through the AssetController and provides a sophisticated dual-channel (standard T+7 and emergency T+0) redemption mechanism through the RedemptionManager, while also supporting the tokenization of redeemed equity into NFT vouchers.

3.2 Vulnerability Information

The following is the status of the vulnerabilities found in this audit:

NO	Title	Category	Level	Status
N1	Contract Bricking when Total Assets is Zero	Design Logic Audit	Information	Acknowledged
N2	Potential DoS in AssetController via Unbounded Asset Loop	Denial of Service Vulnerability	Low	Acknowledged
N3	Potential Share Price Manipulation via Read-Only Reentrancy	Reentrancy Vulnerability	Information	Acknowledged
N4	Indefinite Fund Locking via lockedMintAssets	Authority Control Vulnerability Audit	Medium	Acknowledged
N5	Missing Output Verification in Asset Operations (OTC & SWAP)	Design Logic Audit	High	Fixed
N6	Unenforced Portfolio Strategy (Layer Ratios)	Design Logic Audit	Medium	Acknowledged
N7	Asset Removal/Deactivation Without Balance Check Leads to Fund Freezing and NAV Crash	Design Logic Audit	High	Fixed
N8	Stale NAV via Missing Cache Invalidation on Configuration Changes	Design Logic Audit	Medium	Fixed
N9	Unrestricted Liquidation Authority	Authority Control Vulnerability Audit	High	Fixed
N10	Unsafe Oracle Precision Assumption With Layer Value Mismatch	Unsafe External Call Audit	Medium	Fixed
N11	NAV Inflation via Double Counting	Design Logic Audit	High	Fixed
N12	Missing On-Chain Verification for Delayed Settlements	Design Logic Audit	High	Fixed
N13	Redundant Code	Others	Suggestion	Fixed

NO	Title	Category	Level	Status
N14	Denial of Service due to Hard Dependency	Denial of Service Vulnerability	Low	Fixed
N15	Missing Liability Rollover Mechanism (Vanishing Liability)	Design Logic Audit	High	Fixed
N16	Unsafe Liability Deduction in _removeLiability	Design Logic Audit	Medium	Fixed
N17	Indefinite Fund Lock via Pending Approval	Design Logic Audit	Information	Acknowledged
N18	Missing Role Assignment for Liquidation	Authority Control Vulnerability Audit	Low	Acknowledged
N19	Inflexible Voucher Threshold Configuration	Design Logic Audit	Low	Fixed
N20	Misleading Settlement Estimate in Previews	Design Logic Audit	Low	Fixed
N21	Data Inconsistency in Liability Adjustments	Design Logic Audit	Medium	Fixed
N22	Misleading Voucher Face Value due to Fee Discrepancy	Design Logic Audit	Low	Fixed
N23	Unbounded Settlement Delay by Keeper	Design Logic Audit	Medium	Fixed
N24	Risk of Excessive Privilege	Authority Control Vulnerability Audit	High	Acknowledged
N25	Fixed-Liability Redemption Model Creates "Bank Run" Incentive	Others	Information	Acknowledged
N26	Token Compatibility Reminder	Others	Information	Acknowledged
N27	Potential DoS in AssetController via Unbounded Asset Loop	Denial of Service Vulnerability	Low	Acknowledged

NO	Title	Category	Level	Status
N28	NAV Manipulation via Unchecked expectedUsdt Input	Design Logic Audit	Low	Acknowledged
N29	Griefing and Settlement Breaking via Public checkAndMarkAbnormal	Design Logic Audit	Medium	Fixed
N30	Missing parameters check	Others	Suggestion	Acknowledged
N31	Missing the event records	Others	Suggestion	Fixed
N32	External Call Reminder	Unsafe External Call Audit	Information	Acknowledged
N33	Broken Slippage Protection in Purchase Logic	Design Logic Audit	Low	Fixed
N34	Inconsistent Slippage Configuration in Asset Sale	Design Logic Audit	Low	Fixed
N35	Logic Order Error leads to Locked Funds Failure	Design Logic Audit	Information	Acknowledged
N36	Rollover Logic Gas Limit Risk (DoS)	Denial of Service Vulnerability	Information	Acknowledged
N37	Share Price Manipulation	Design Logic Audit	High	Fixed
N38	NAV Crash via Liability Mismanagement	Design Logic Audit	High	Fixed
N39	Rigid Historical Value Check in Settlement (DoS Risk)	Denial of Service Vulnerability	Information	Acknowledged

4 Code Overview

4.1 Contracts Description

Audit Version:

https://github.com/Paimon-R-D/paimon-bsc-contracts

commit: cb9bf04686c8e1c210466da1b527db6af75ca3d3

Fixed Version:

commit: 9883ad74701445d7449d60b252488b92e4f075d4

Audit scope:

```
└── src
    ├── ppt
    │   ├── AssetController.sol
    │   ├── IPPTContracts.sol
    │   ├── PPT.sol
    │   ├── PPTTypes.sol
    │   ├── RedemptionManager.sol
    │   └── RedemptionVoucher.sol
    └── Counter.sol
```

The main network address of the contract is as follows:

Contract Address(BSC)		
Contract Name	Contract Address (Proxy)	Contract Address (Implementation)
PP Token (PP)	0x8505c32631034A7cE8800239c08547e0434EdaD9	0x3D3a244bdb90C19680961645afc27312705DA0FA
RedemptionVoucher	0x73F42b0D657785fE844e3BF486Fe1e15fFE13514	0xb78D182702701cf1Abc5a7ACD93Fc2c67d43719F
AssetController	0x6F5170956132588E9b2844478f1fF1B387573A3D	0xB57A2327FAC0547D4F9de7265e07C65E6E3261E6
RedemptionManager	0xd614a6fe8C35aC9af4F59cd14849877179cDCdB9	0xDB5b2dF4C2a7AC118a1De3CB9d13eD5aCfF01c1F

Role Address(BSC)		
Role Name	Role Bytes	Role Address
DEFAULT_ADMIN_ROLE	0x0000000000000000000000000000000000000000000000000000000000000000	0xDc814FdF8F5b969C29257e434C8205678a03f810 (4 out of 5 signers Multisig)

Role Address(BSC)		
ADMIN_ROLE	0xa49807205ce4d355092ef5a8a18f56e8913cf4a201fbe287825b095693c21775	0xDc814FdF8F5b969C29257e434C8205678a03f810 (4 out of 5 signers Multisig)
UPGRADER_ROLE	0x189ab7a9244df0848122154315af71fe140f3db0fe014031783b0946b8c9d2e3	0x2573d3c585fD4821353Def01E2fb69f3385f7A59 (TimelockController)
KEEPER_ROLE	0xfc8737ab85eb45125971625a9ebdb75cc78e01d5c1fa80c4c6e5203f47bc4fab	0xbc6f07125A515bA1200C61E31B00250BE5EFcf5c (EOA)
REBALANCER_ROLE	0xccc64574297998b6c3edf6078cc5e01268465ff116954e3af02ff3a70a730f46	0xfA211f036f8b94982a54c20D734e40bb29b56666 (EOA)
OPERATOR_ROLE	0x97667070c54ef182b0f5858b034beac1b6f3089aa2d3188bb1e8929f4fa9b929	0xd614a6fe8C35aC9af4F59cd14849877179cDCdB9(RedemptionManager) /0x6F5170956132588E9b2844478f1fF1B387573A3D (AssetController)
LIQUIDATOR_ROLE	0x5e17fc5225d4a099df75359ce1f405503ca79498a8dc46a7d583235a0ee45c16	0xd614a6fe8C35aC9af4F59cd14849877179cDCdB9(RedemptionManager)

4.2 Visibility Description

The SlowMist Security team analyzed the visibility of major contracts during the audit, the result as follows:

AssetController			
Function Name	Visibility	Mutability	Modifiers
<Constructor>	Public	Can Modify State	-
initialize	External	Can Modify State	initializer
_authorizeUpgrade	Internal	Can Modify State	onlyRole
addAsset	External	Can Modify State	onlyRole
removeAsset	External	Can Modify State	onlyRole

AssetController			
updateAssetConfig	External	Can Modify State	onlyRole
setAssetActive	External	Can Modify State	onlyRole
purchaseAsset	External	Can Modify State	onlyRole nonReentrant whenNotPaused
redeemAsset	External	Can Modify State	onlyRole nonReentrant whenNotPaused
executeWaterfallLiquidation	External	Can Modify State	onlyRole nonReentrant
setLayerConfig	External	Can Modify State	onlyRole
getLayerConfigs	External	-	-
validateLayerRatios	Public	-	-
withdrawRedemptionFees	External	Can Modify State	onlyRole
getAssetConfigs	External	-	-
getLayerAssets	External	-	-
getLayerValue	Public	-	-
calculateAssetValue	Public	-	-
calculateAssetValueFresh	Public	Can Modify State	-
refreshCache	External	Can Modify State	-
_invalidateCache	Internal	Can Modify State	-
_calculateAssetValueInternal	Internal	-	-
_asset	Internal	-	-
_totalAssets	Internal	-	-
_previewDeposit	Internal	-	-

AssetController			
_executePurchase	Internal	Can Modify State	-
_sellAsset	Internal	Can Modify State	-
_executeWaterfallLiquidation	Internal	Can Modify State	-
_liquidateAsset	Internal	Can Modify State	-
setOracleAdapter	External	Can Modify State	onlyRole
setSwapHelper	External	Can Modify State	onlyRole
setDefaultSwapSlippage	External	Can Modify State	onlyRole
setDelayedSettlementAsset	External	Can Modify State	onlyRole
setDefaultSettlementTimeout	External	Can Modify State	onlyRole
redeemDelayedAsset	External	Can Modify State	onlyRole nonReentrant whenNotPaused
purchaseDelayedAsset	External	Can Modify State	onlyRole nonReentrant whenNotPaused
confirmSettlement	External	Can Modify State	onlyRole
checkAndMarkAbnormal	External	Can Modify State	-
pause	External	Can Modify State	onlyRole
unpause	External	Can Modify State	onlyRole

Counter			
Function Name	Visibility	Mutability	Modifiers
setNumber	Public	Can Modify State	-
increment	Public	Can Modify State	-

Counter			

RedemptionVoucher			
Function Name	Visibility	Mutability	Modifiers
<Constructor>	Public	Can Modify State	-
initialize	External	Can Modify State	initializer
_authorizeUpgrade	Internal	Can Modify State	onlyRole
mint	External	Can Modify State	onlyRole
burn	External	Can Modify State	onlyRole
voucherInfo	External	-	-
requestToToken	External	-	-
getVoucherByRequest	External	-	-
isSettleable	External	-	-
timeUntilSettlement	External	-	-
tokenURI	Public	-	-
_generateSVG	Internal	-	-
_formatAmount	Internal	-	-
_update	Internal	Can Modify State	-
_increaseBalance	Internal	Can Modify State	-
supportsInterface	Public	-	-

RedemptionManager			
Function Name	Visibility	Mutability	Modifiers
<Constructor>	Public	Can Modify State	-

RedemptionManager			
initialize	External	Can Modify State	initializer
_authorizeUpgrade	Internal	Can Modify State	onlyRole
requestRedemption	External	Can Modify State	nonReentrant whenNotPaused
requestEmergencyRedemption	External	Can Modify State	nonReentrant
settleRedemption	External	Can Modify State	whenNotPaused nonReentrant
settleWithVoucher	External	Can Modify State	whenNotPaused nonReentrant
_settleRedemptionCore	Internal	Can Modify State	-
cancelRedemption	External	Can Modify State	nonReentrant
previewRedemption	External	-	-
previewEmergencyRedemption	External	-	-
approveRedemption	External	Can Modify State	onlyRole
approveRedemptionWithDate	External	Can Modify State	onlyRole
_approveRedemptionCore	Internal	Can Modify State	-
rejectRedemption	External	Can Modify State	onlyRole
_processStandardRedemption	Internal	Can Modify State	-
_processEmergencyRedemption	Internal	Can Modify State	-
_executeSettlement	Internal	Can Modify State	-
_requiresStandardApproval	Internal	-	-
_requiresEmergencyApproval	Internal	-	-

RedemptionManager			
_calculateRedemptionFee	Internal	-	-
_balanceOf	Internal	-	-
_asset	Internal	-	-
_totalAssets	Internal	-	-
_addToPendingApprovals	Internal	Can Modify State	-
_removeFromPendingApprovals	Internal	Can Modify State	-
getRedemptionRequest	External	-	-
getUserRedemptions	External	-	-
getPendingApprovals	External	-	-
getTotalPendingApprovalAmount	External	-	-
getRequestCount	External	-	-
_getDayIndex	Internal	-	-
_addDailyLiability	Internal	Can Modify State	-
_removeLiability	Internal	Can Modify State	-
getSevenDayLiability	Public	-	-
getOverdueLiability	Public	-	-
getDailyLiability	Public	-	-
getOverdueLiability	External	-	-
adjustOverdueLiability	External	Can Modify State	onlyRole
adjustDailyLiability	External	Can Modify State	onlyRole
setVoucherThreshold	External	Can Modify State	onlyRole

RedemptionManager			
setAssetController	External	Can Modify State	onlyRole
setBaseRedemptionFee	External	Can Modify State	onlyRole
setEmergencyPenaltyFee	External	Can Modify State	onlyRole
setStandardApprovalAmount	External	Can Modify State	onlyRole
setStandardApprovalQuotaRatio	External	Can Modify State	onlyRole
setEmergencyApprovalAmount	External	Can Modify State	onlyRole
setEmergencyApprovalQuotaRatio	External	Can Modify State	onlyRole
pause	External	Can Modify State	onlyRole
unpause	External	Can Modify State	onlyRole

PPT			
Function Name	Visibility	Mutability	Modifiers
<Constructor>	Public	Can Modify State	-
initialize	External	Can Modify State	initializer
_authorizeUpgrade	Internal	Can Modify State	onlyRole
totalAssets	Public	-	-
effectiveSupply	Public	-	-
sharePrice	Public	-	-
grossAssets	Public	-	-
_getGrossAssets	Internal	-	-
_convertToShares	Internal	-	-

PPT			
_convertToAssets	Internal	-	-
deposit	Public	Can Modify State	nonReentrant whenNotPaused
mint	Public	Can Modify State	nonReentrant whenNotPaused
withdraw	Public	-	-
redeem	Public	-	-
getLayer1Liquidity	Public	-	-
getLayer1Cash	Public	-	-
getLayer1YieldAssets	Public	-	-
getLayer2Liquidity	Public	-	-
getLayer3Value	Public	-	-
getAvailableLiquidity	Public	-	-
getRedeemableLiquidity	Public	-	-
getStandardChannelQuota	Public	-	-
getVaultState	External	-	-
getLiquidityBreakdown	External	-	-
lockShares	External	Can Modify State	onlyOperator
unlockShares	External	Can Modify State	onlyOperator
burnLockedShares	External	Can Modify State	onlyOperator
addRedemptionLiability	External	Can Modify State	onlyOperator
removeRedemptionLiability	External	Can Modify State	onlyOperator
addRedemptionFee	External	Can Modify State	onlyOperator
reduceRedemptionFee	External	Can Modify State	onlyOperator

PPT			
transferAssetTo	External	Can Modify State	onlyOperator
getAssetBalance	External	-	-
approveAsset	External	Can Modify State	onlyOperator
reduceEmergencyQuota	External	Can Modify State	onlyOperator
restoreEmergencyQuota	External	Can Modify State	onlyOperator
addPendingApprovalShares	External	Can Modify State	onlyOperator
removePendingApprovalShares	External	Can Modify State	onlyOperator
convertPendingToLocked	External	Can Modify State	onlyOperator
setAssetController	External	Can Modify State	onlyRole
refreshEmergencyQuota	External	Can Modify State	onlyRole
resetLockedMintAssets	External	Can Modify State	onlyOperator
setRedemptionManager	External	Can Modify State	onlyRole
setStandardQuotaRatio	External	Can Modify State	onlyRole
pause	External	Can Modify State	onlyRole
unpause	External	Can Modify State	onlyRole

4.3 Vulnerability Summary

[N1] [Information] Contract Bricking when Total Assets is Zero

Category: Design Logic Audit

Content

In the PPT contract, both _convertToShares and _convertToAssets functions enforce a strict check that totalAssets() must be greater than zero if effectiveSupply() is greater than zero. If the vault suffers a significant loss such that grossAssets roughly equals totalRedemptionLiability (net totalAssets() ≈ 0), or if the vault is completely drained

(100% loss), the contract becomes permanently bricked. In this state, neither deposit (via previewDeposit) nor mint (via _convertToAssets) can succeed. The contract enters a deadlock state where it is insolvent and refuses any new capital that could dilute the existing shares to a non-zero value. This effectively freezes the protocol, preventing any recapitalization through standard methods. Recovery Note: The only way to unbrick the contract is to perform a direct ERC20 transfer (donation) of underlying assets to the vault address, sufficient to cover the totalRedemptionLiability. Once grossAssets > totalRedemptionLiability, totalAssets() becomes positive, and the contract resumes normal operation.

Code location:

src/ppt/PPT.sol#L218-L236

```
    function _convertToShares(uint256 assets, Math.Rounding rounding) internal view
virtual override returns (uint256) {
        ..
        if(supply>0){
            require(totalAsset>0, "totalAsset is not 0");
        }
        ...
    }

    function _convertToAssets(uint256 shares, Math.Rounding rounding) internal view
virtual override returns (uint256) {
        ...
        if(supply>0){
            require(totalAsset>0, "totalAsset is not 0");
        }
        ...
    }
```

Solution

It's recommended to remove the explicit require(totalAsset > 0) check OR implement a specific rescueDeposit function that bypasses this check to allow recapitalization during insolvency. If keeping the check is desired for user protection, the UI and documentation must clearly state that insolvency results in a permanent freeze until external debt coverage is provided.

Status

Acknowledged

[N2] [Low] Potential DoS in AssetController via Unbounded Asset Loop

Category: Denial of Service Vulnerability

Content

The calculateAssetValue function in AssetController iterates through all configured assets to calculate the total value. There is no hard limit on the number of assets that can be added to the _assetConfigs array. If too many assets are added, or if specific asset balanceOf / decimals calls are expensive (or malicious), this loop could exceed the block gas limit. Since deposit and mint functions in PPT contract force a call to calculateAssetValueFresh() (which executes this loop via _calculateAssetValueInternal), this would freeze all deposits and redemptions effectively causing a Denial of Service.

Code location:

src/ppt/AssetController.sol#L622-L649

```
    function _calculateAssetValueInternal() internal view returns (uint256 totalValue)
{
        ...
        for (uint256 i = 0; i < _assetConfigs.length; i++) {
            ...
            totalValue += value;
        }
        totalValue += totalPendingValue;
    }
```

Solution

It's recommended to implement a strict limit on the maximum number of active assets that can be added.

Status

Acknowledged

[N3] [Information] Potential Share Price Manipulation via Read-Only Reentrancy

Category: Reentrancy Vulnerability

Content

The PPT contract is susceptible to read-only reentrancy attacks, particularly during asset purchase and redemption operations in AssetController. Although purchaseAsset and redeemAsset are protected by nonReentrant modifier preventing state changes, the view functions like totalAssets() and sharePrice() are not. During the execution of

purchaseAsset or settleRedemption (via executeWaterfallLiquidation), funds are transferred out of the Vault to the external adapter or swapHelper before the expected assets are received. If the adapter or the underlying DEX (e.g., Uniswap hooks, or a malicious adapter) allows for a callback or control transfer during the swap, an attacker can call PPT.totalAssets() or sharePrice(). At this specific moment, the Vault's cash balance (cashValue) has decreased, but the new asset balance (assetValue) has not yet increased (or the oracle has not updated to reflect it). This causes totalAssets() to under-report the actual value, leading to a temporary, artificial drop in share price. An attacker can exploit this via a flash loan to manipulate the price and profit from third-party protocols (e.g., lending markets, derivative platforms) that rely on PPT's sharePrice() as an oracle.

Scenario A: Purchase Asset (purchaseAsset)

1.AssetController calls purchaseAdapter.

2.Funds transferred OUT of Vault (Cash drops).

3.Adapter triggers callback (e.g. malicious or via DEX hook).

4.Attacker calls AssetController.calculateAssetValueFresh() or refreshCache(), then call the sharePrice() -> Price crashes.

5.Assets finally received.

Scenario B: Redemption Settlement (settleRedemption)

1.RedemptionManager calls executeWaterfallLiquidation due to insufficient cash.

2.AssetController calls redeemAsset to sell investment assets.

3.Investment asset transferred OUT (Asset Balance drops).

4.Adapter/DEX triggers callback before sending USDT back.

5.Attacker calls AssetController.refreshCache() to force update NAV to exclude the sold asset, then calls sharePrice() -> Price crashes (because investment asset is gone, but cash hasn't arrived).

Code location:

src/ppt/AssetController.sol#L772-L774, L801, L833-L835, L846-L848

```
            (bool success,) = config.purchaseAdapter.call(
                 abi.encodeWithSignature("purchase(address,uint256)", address(vault),
amountIn)
             );
```

```
            tokensReceived = swapHelper.buyRWAAsset(tokenIn, config.tokenAddress,
amountIn, slippageBps, address(vault));
```

Solution

Do not rely on PPT.sharePrice() as a spot oracle in a context where reentrancy is possible. And restrict `calculateAssetValueFresh()` and `refreshCache()` functions to trusted roles only (e.g., Keeper, Admin).

It's also recommended to implement a "Rebalancing" flag in the AssetController contract. When `purchaseAsset` or `executeWaterfallLiquidation` is executing, set this flag to `true`. `PPT.sharePrice()` and `totalAssets()` should revert if `AssetController.isRebalancing()` is true.

Status

Acknowledged

[N4] [Medium] Indefinite Fund Locking via lockedMintAssets

Category: Authority Control Vulnerability Audit

Content

The lockedMintAssets variable in the PPT contract accumulates the value of all deposits and mints. This value is subtracted from the liquid cash when calculating getRedeemableLiquidity in AssetController, effectively preventing these funds from being used to satisfy redemptions. However, lockedMintAssets is only reset to zero when AssetController.purchaseAsset is called by the REBALANCER_ROLE. There is no time-based unlock mechanism (e.g., a cooldown period). If the REBALANCER_ROLE decides not to purchase new assets for an extended period (or loses the key), the deposited funds remain "locked" indefinitely in the lockedMintAssets counter. In the _executePurchase function, the contract checks `if (amountIn > availableBalance)` where `availableBalance` is calculated by deducting `lockedMintAssets` from the physical balance. If `lockedMintAssets` accumulates to a value larger than the physical balance (e.g., due to accounting errors or extreme scenarios where no purchases happen for a long time), `availableBalance` becomes 0. The `purchaseAsset` function (which is the only standard way to reset `lockedMintAssets`) will revert. This means that even if the vault holds sufficient cash, it will report 0 redeemable liquidity for that portion, preventing users from redeeming their shares against those specific funds. This creates a centralization risk where user liquidity is contingent on active management actions.

Code location:

src/ppt/PPT.sol#L529-L533

src/ppt/AssetController.sol#L394-L405

```
    function resetLockedMintAssets() external override onlyOperator{
        uint256 old = lockedMintAssets;
        lockedMintAssets = 0;
        emit LockedMintAssetsReset(old);
    }

    function purchaseAsset(
        address tokenIn,
        address tokenOut,
        uint256 amountIn
    ) external override onlyRole(REBALANCER_ROLE) nonReentrant whenNotPaused returns
(uint256 tokensReceived) {
        ...
        vault.resetLockedMintAssets();
        ...
    }
```

Solution

It's recommended to implement a time-based decay or unlock mechanism for lockedMintAssets.

Status

Acknowledged; After communicating with the project team, they stated that this design is released by our backend program. Due to business needs, some asset purchase windows in RWA are not always available, so a portion of the reserved assets will be locked.

[N5] [High] Missing Output Verification in Asset Operations (OTC & SWAP)

Category: Design Logic Audit

Content

The AssetController lacks independent verification of trade outcomes for both OTC and SWAP modes.

OTC Mode: Logic calculates received amounts via balanceOf difference (which is good), but fails to enforce any minimum output amount (no slippage protection). The contract accepts whatever the adapter sends, even if it's 1 wei.

SWAP Mode: Logic passes a slippageBps to swapHelper, but blindly trusts the return value of the helper. It does not verify the actual token balance increase of the Vault.

If swapHelper is malicious, buggy, or interacts with a fee-on-transfer token, the tokensReceived return value might differ from the actual assets arriving in the Vault.

AssetController does not compare the execution price against the valid Oracle price to ensure the trade was executed within market bounds.

Code location:

src/ppt/AssetController.sol#L757-L803, L830-L853

```
            (bool success,) = config.purchaseAdapter.call(
                 abi.encodeWithSignature("purchase(address,uint256)", address(vault),
amountIn)
             );

            tokensReceived = swapHelper.buyRWAAsset(tokenIn, config.tokenAddress,
amountIn, slippageBps, address(vault));
```

Solution

For both modes, check the actual balanceOf change in the Vault.

Min Amount Check: require(actualReceived >= minOutputAmount).

minOutputAmount should be calculated using the Oracle Price minus a maximum allowed tolerance (maxSlippage), ensuring the trade happens near fair market value regardless of the execution method.

Status

Fixed

[N6] [Medium] Unenforced Portfolio Strategy (Layer Ratios)

Category: Design Logic Audit

Content

The AssetController defines a sophisticated LayerConfig system with Target, Minimum, and Maximum ratios for each liquidity tier (Cash, MMF, HYD). However, these constraints are never enforced in any operational function.

purchaseAsset: accessible by REBALANCER_ROLE, does not check if the purchase pushes the tier above maxRatio.

redeemAsset: accessible by REBALANCER_ROLE, does not check if the redemption pushes the tier below minRatio.

updateAssetConfig: accessible by KEEPER_ROLE, allows moving an asset (with non-zero balance) from one tier to another. This could immediately violate the destination tier's maxRatio or the source tier's minRatio.

setLayerConfig: accessible by KEEPER_ROLE, allows setting target/min/max ratios for a single layer. Crucially, it does not check if the sum of targetRatio across all layers equals 100% (or even if min/max ranges overlap logically between layers). The validateLayerRatios function exists but is public view and never called internally. This renders the on-chain governance of portfolio strategy purely ornamental. A REBALANCER or KEEPER can bypass the strategy, or accidentally create a mathematically impossible strategy (e.g. total target = 0%).

Code location:

src/ppt/AssetController.sol#L390-L410 (purchaseAsset)

src/ppt/AssetController.sol#L417-L436 (redeemAsset)

src/ppt/AssetController.sol#L464-L480 (setLayerConfig - missing validation)

Solution

In setLayerConfig, require(validateLayerRatios()) or at least emit a warning event if invalid.

In purchaseAsset and redeemAsset, require post-trade ratios respect the limits.

Status

Acknowledged; After communicating with the project team, they stated that because this is a net asset value investment, the asset price fluctuations vary, making it impossible to fully adhere to this allocation ratio.

[N7] [High] Asset Removal/Deactivation Without Balance Check Leads to Fund Freezing and NAV Crash

Category: Design Logic Audit

Content

The removeAsset and setAssetActive functions allow the KEEPER_ROLE to remove or deactivate an asset configuration without checking if the vault's balance for that asset is zero. If an asset with a non-zero balance is removed or deactivated (isActive = false).

NAV Crash: The asset's value is immediately excluded from _calculateAssetValueInternal (which loops through _assetConfigs and skips inactive ones). This causes totalAssets() to drop instantly by the value of the held assets.

Fund Freezing: The assets remain in the Vault wallet, but the AssetController logic ignores them. redeemAsset and purchaseAsset revert if called for a disabled/removed token. The funds become effectively frozen.

Liquidity Crisis: The asset is skipped during waterfall liquidation.

Code location:

src/ppt/AssetController.sol#L288-L313 (removeAsset)

src/ppt/AssetController.sol#L365-L377 (setAssetActive)

```
function removeAsset(address token) external override onlyRole(KEEPER_ROLE) {
    ...
}

function setAssetActive(
    address token,
    bool active
) external override onlyRole(KEEPER_ROLE) {
    ...
}
```

Solution

It's recommended to add a strict check of the asset token balanceOf(address(vault)) == 0 in the setAssetActive and removeAsset functions.

Status

Fixed

[N8] [Medium] Stale NAV via Missing Cache Invalidation on Configuration Changes

Category: Design Logic Audit

Content

The AssetController relies on a caching mechanism (_cachedAssetValue) to return the total asset value efficiently. This cache is valid for CACHE_DURATION (e.g., 5 minutes). However, administrative functions that alter the set of assets included in the valuation—specifically addAsset, removeAsset, and setAssetActive—do not invalidate this cache. This leads to a state where the contract configuration has changed, but the reported NAV reflects the old configuration until the cache naturally expires or is forced refreshed.

setAssetActive(false) / removeAsset: If an asset is disabled or removed, it is immediately excluded from _calculateAssetValueInternal. However, calculateAssetValue() will return the old (higher) cached value. Users can redeem at this inflated price, draining value from the vault.

addAsset / setAssetActive(true): If an asset is added or re-activated (especially if the vault already holds a balance of

it), calculateAssetValue() will return the old (lower) cached value. New depositors can mint shares at a discount (arbitrage), effectively stealing value from existing shareholders once the cache creates the NAV jump.

Code location:

src/ppt/AssetController.sol#L255 (addAsset - missing _invalidateCache) src/ppt/AssetController.sol#L365 (setAssetActive - missing _invalidateCache) src/ppt/AssetController.sol#L288 (removeAsset - missing _invalidateCache)

Solution

It's recommended to add _invalidateCache() (or _cachedAssetValue.timestamp = 0) to the end of addAsset, removeAsset, and setAssetActive functions to ensure NAV is immediately recalculated upon any configuration change.

Status

Fixed

[N9] [High] Unrestricted Liquidation Authority

Category: Authority Control Vulnerability Audit

Content

The executeWaterfallLiquidation function allows the REBALANCER_ROLE to liquidate vault assets to generate USDT. However, it blindly accepts the amountNeeded parameter without any validation against the actual protocol state (e.g., PPT.totalRedemptionLiability).

Scenario:

A compromised or malicious REBALANCER_ROLE (or operator) can call executeWaterfallLiquidation(uint256.max, ...) directly. The _liquidateAsset logic is "Fill or Kill". It loops through all assets in TIER_1 and TIER_2. Because `tokenValue < type(uint256).max`, the logic `if (tokenValue <= amountNeeded) tokensToSell = balance` triggers for every single asset.

The function systematically sells 100% of the vault's portfolio (excluding delayed assets) into USDT via the configured Adapters or SwapHelper.

This creates massive market impact (slippage), incurs huge fees, destroys the portfolio structure, and effectively converts the RWA protocol into a cash pile (or worse, drains it if the Swap path is malicious).

Code location:

src/ppt/AssetController.sol#L443-L451

```
    function executeWaterfallLiquidation(
        uint256 amountNeeded,
        PPTTypes.LiquidityTier maxTier
    ) external override onlyRole(REBALANCER_ROLE) nonReentrant returns (uint256
funded) {
        funded = _executeWaterfallLiquidation(amountNeeded, maxTier);

        // Invalidate cache after liquidation
        _invalidateCache();
    }
```

Solution

It's recommended to add a check in the executeWaterfallLiquidation function, read PPT.totalRedemptionLiability() and enforce require(amountNeeded <= vault.totalRedemptionLiability() - vault.cash(), "Excessive liquidation"). And restrict executeWaterfallLiquidation to be callable only by the RedemptionManager contract (via an onlyRedemptionManager modifier), removing the ability for human operators (REBALANCER_ROLE) to call it directly with arbitrary amounts.

Status

Fixed; In the fixed version, the project team commented out the code logic for TIER_2_MMF that would normally be liquidated. This means that now only TIER_1_CASH assets can be liquidated by REBALANCER_ROLE through the `executeWaterfallLiquidation` function. Furthermore, in the project's settings, REBALANCER_ROLE is an EOA address used to execute asset purchases/sales/liquidations.

[N10] [Medium] Unsafe Oracle Precision Assumption With Layer Value Mismatch

Category: Unsafe External Call Audit

Content

The system contains a critical reliance on the Oracle Adapter returning prices by default with 18 decimals, which is not validated or enforced.

1.The _calculateAssetValueInternal function hardcodes logic like `value / (10 ** (18 - baseDecimals))`. If the Oracle returns 8 decimals (common for Chainlink USD feeds), the calculation Price(1e8) * Balance / 10**12 will result

in zero (underflow/truncation) for most assets. The Vault's Total Assets will report 0, causing a price crash or deadlock.

2.The getLayerValue calculates `(balance * price) / 10**decimals` but skips strict normalization to the Vault's base decimals. Even if the Oracle returns 18 decimals, the getLayerValue function returns a result in 18 decimals. The PPT contract incorrectly compares this 18-decimal value against 6-decimal Quotas (if Vault is 6 decimals) disabling the quota system.

3.The _liquidateAsset tokenValue is calculated as `(balance * price) / 10**decimals` (assuming Oracle is 18). The amountNeeded is passed from RedemptionManager in Base Asset Precision.

The check if (tokenValue <= amountNeeded) compares with different decimals, leading to incorrect branching. The calculation `tokensToSell = (amountNeeded * (10 ** decimals)) / price incorrectly mixes precisions (Base * Asset(X) / Oracle(18))`, resulting in massive calculation errors.

Code location:

src/ppt/AssetController.sol#L573 (getLayerValue missing normalization)

src/ppt/AssetController.sol#L638

src/ppt/AssetController.sol#L906-L912 (_liquidateAsset precision mismatch)

```
            if (baseDecimals < 18) {
                value = value / (10 ** (18 - baseDecimals));
            }
```

Solution

It’s recommended to ensure OracleAdapter always normalizes prices to 18 decimals internally before returning, retrieve oracle.decimals() dynamically or enforce the underlying oracle's precision in the Adapter config, and apply the same normalization logic as _calculateAssetValueInternal (target baseDecimals) and _liquidateAsset functions so PPT receives the correct unit.

Status

Fixed; The fix version introduced PaimonOracleAdapter to enforce that the oracle returns a uniform 18-bit precision price. However, the getLayerValue and _liquidateAsset functions lack precision checks on assets and do not perform such checks when adding them.

[N11] [High] NAV Inflation via Double Counting

Category: Design Logic Audit

Content

The protocol's accounting logic is fundamentally flawed because it treats totalPendingValue and balanceOf (Physical Assets) as purely additive components, failing to enforce mutual exclusion during state transitions.

Creation Phase Error (No Deduction):

Purchase: purchaseDelayedAsset adds expectedTokenValue to totalPendingValue immediately. It does not deduct the corresponding USDT amount from the Vault's accounted Cash. eg.: NAV = Cash (Held) + Pending Token.

Redeem: redeemDelayedAsset adds expectedUsdt to totalPendingValue immediately.

It does not deduct the corresponding Asset Balance (especially if _sellAsset relies on delayed settlement and retains the token). eg.: NAV = Asset (Held) + Pending Cash.

The same capital is counted twice in both directions.

Settlement Phase Error (No Automatic Offset):

When funds (USDT or Asset) physically arrive, balanceOf updates immediately, increasing NAV.

The system has no mechanism to automatically reduce totalPendingValue upon this arrival. It relies on a separate, manual confirmSettlement. Like: NAV = Asset (Arrived) + Pending Value (Stale). The same capital is counted twice (as held asset and as pending asset).

In all cases, grossAssets is artificially inflated by the pending amount. Attackers can exploit this 100% predictable inflation to arbitrage share price or redeem at elevated values.

Code location:

src/ppt/AssetController.sol#L644-L648 (Double Addition)

src/ppt/AssetController.sol#L997-L1077

```
    function _calculateAssetValueInternal() internal view returns (uint256 totalValue)
{
        ...
        for (uint256 i = 0; i < _assetConfigs.length; i++) {
            ...
            totalValue += value;
        }
        totalValue += totalPendingValue;
    }
```

```
    function redeemDelayedAsset(
        address token,
        uint256 tokenAmount,
        uint256 expectedUsdt
    ) external override onlyRole(REBALANCER_ROLE) nonReentrant whenNotPaused returns
(uint256 settlementId) {
        ...
        if (success && usdtReceived > 0) {
            emit AssetRedeemed(token, config.tier, tokenAmount, usdtReceived);
            _invalidateCache();
            return 0; // No pending settlement needed
        }
        ...
        totalPendingValue += expectedUsdt;

        emit SettlementCreated(settlementId, PPTTypes.SettlementType.SALE, token,
tokenAmount, expectedUsdt);
        _invalidateCache();
    }

    function purchaseDelayedAsset(
        address token,
        uint256 usdtAmount,
        uint256 expectedTokenValue
    ) external override onlyRole(REBALANCER_ROLE) nonReentrant whenNotPaused returns
(uint256 settlementId) {
        ...
        totalPendingValue += expectedTokenValue;

        emit SettlementCreated(settlementId, PPTTypes.SettlementType.PURCHASE, token,
usdtAmount, expectedTokenValue);
        _invalidateCache();
    }
```

Solution

The logic must ensure that when Pending increases, Balance/Cash decreases (Atomic Transfer/Burn). Upon settlement, effectively ensure that the Arrived Balance is not counted until the Pending Value is removed, or force confirmSettlement to happen atomically with the arrival. Or modify _calculateAssetValueInternal to deduct pendingSettlements[i].expectedValue from the balanceOf it replaces, preventing the overlap.

Status

Fixed; In the fix version, this issue is resolved by implementing Controller-Initiated Accounting. The purchaseDelayedAsset function has been removed from the public interface. Instead, asset purchases and their

corresponding pending values are now atomically created by the Controller (under REBALANCER_ROLE control) before any funds leave the vault.

[N12] [High] Missing On-Chain Verification for Delayed Settlements

Category: Design Logic Audit

Content

The functions purchaseDelayedAsset, confirmSettlement, and checkAndMarkAbnormal operate as a purely trust-based bookkeeping system with no on-chain verification of actual fund transfers.

No Proof of Purchase:

purchaseDelayedAsset creates a pending settlement record and increases totalPendingValue (inflating NAV). However, it does not verify that the REBALANCER actually sent the usdtAmount out of the Vault. A malicious Rebalancer can spam this function to pump the NAV indefinitely without moving a single wei of funds.

No Proof of Settlement:

confirmSettlement marks a settlement as complete and removes the pending value. It does not verify that the expected assets actually arrived in the Vault (via balance check). A compromised Keeper could confirm fake settlements, effectively "cleaning" the fraudulent pending records created in step 1, leaving the Vault with a permanent hole in its balance sheet.

Unverified Write-offs:

checkAndMarkAbnormal allows writing off pending value if it times out. This can be abused to hide theft (funds sent out, never returned, then written off as "abnormal").

Code location:

src/ppt/AssetController.sol#L1049 (purchaseDelayedAsset - no transfer logic)

src/ppt/AssetController.sol#L1082 (confirmSettlement - no balance check)

src/ppt/AssetController.sol#L1101 (checkAndMarkAbnormal)

Solution

The purchaseDelayedAsset functions must include the logic to transfer USDT to the counterparty (or Adapter) within the same transaction. The confirmSettlement function should require a proof, such as checking that balanceOf(token) increased by expectedTokenValue. Or requiring the settlement transaction to include the funds transfer itself.

Status

Fixed; In the fix version, this issue is resolved by adding On-Chain Verification to the settlement process. The confirmSettlement function (controlled by REBALANCER_ROLE) now actively "pulls" the returned assets from the adapter using transferFrom and verifies the received amount against the Oracle value on-chain.

[N13] [Suggestion] Redundant Code

Category: Others

Content

1.The _executePurchase contains a check if (amountIn == 0) return (0, 0);. However, its only caller, the external function purchaseAsset, already performs a strict check if (amountIn == 0) revert ZeroAmount(); before calling _executePurchase. This makes the internal check unreachable code (dead code), which slightly increases gas cost and code clutter.

Code location:

src/ppt/AssetController.sol#L718

```
        if (amountIn == 0) return (0, 0);
```

2.The following internal function code is not used in the code.

src/ppt/AssetController.sol#L662-L669

src/ppt/RedemptionManager.sol#L797, 807

```
    function _totalAssets() internal view returns (uint256) {
        return IERC4626(address(vault)).totalAssets();
    }

    function _previewDeposit(uint256 assets) internal view returns (uint256) {
        return IERC4626(address(vault)).previewDeposit(assets);
    }

    function _balanceOf(address owner) internal view returns (uint256) {
        return IERC4626(address(vault)).balanceOf(owner);
    }

    function _totalAssets() internal view returns (uint256) {
```

```
        return IERC4626(address(vault)).totalAssets();
    }
```

Solution

It's recommended to remove the redundant code.

Status

Fixed

[N14] [Low] Denial of Service due to Hard Dependency

Category: Denial of Service Vulnerability

Content

The AssetController contract process treats swapHelper and oracleAdapter as strict dependency. In the _executeWaterfallLiquidation function, the code checks: if (address(swapHelper) == address(0)) return 0;. In the _sellAsset function, if purchaseAdapter is not set or fails, it falls back to swapHelper. If swapHelper is not set, it returns failure. If swapHelper is not configured, is paused, or accidentally removed (set to address 0), the entire liquidation system fails. Consequently, RedemptionManager (which relies on executeWaterfallLiquidation to raise cash) will fail to raise funds and revert with InsufficientLiquidity. This creates a Denial of Service (DoS) for all user redemptions that exceed the cash buffer. A single missing configuration bricks the redemption flow. Similarly, the `_liquidateAsset` function relies on the arbitrary `amountNeeded` parameter, while the `_calculateAssetValueInternal` function relies on the previously passed `totalPendingValue` parameter and has no actual calculation reference.

Code location:

src/ppt/AssetController.sol#L868

```
        if (address(swapHelper) == address(0)) return 0;
```

Solution

It's recommended to remove the strict check if (address(swapHelper) == address(0)) return 0; at the start of _executeWaterfallLiquidation. And allow the loop to proceed and try purchaseAdapter for individual assets.

Status

Fixed

[N15] [High] Missing Liability Rollover Mechanism (Vanishing Liability)

Category: Design Logic Audit

Content

The RedemptionManager fails to automatically roll over expired daily liabilities into `overdueLiability`, creating a gap in the quota calculation. The getSevenDayLiability functions sums dailyLiability from `today` to `today + 7`. The getOverdueLiability functions returns the static overdueLiability variable. But there is no function (cron or trigger) that moves dailyLiability[yesterday] into overdueLiability. As time passes, a redemption scheduled for "Yesterday" (which was not processed) drops out of the "7 Day" window. Since it is not added to "Overdue", it disappears entirely from getStandardChannelQuota. The getStandardChannelQuota function effectively under-reports the liability, allowing new redemptions to bypass the intended maxAvailable cap, potentially draining liquidity meant for the overdue requests.

Code location:

src/ppt/RedemptionManager.sol#L902 (getSevenDayLiability - starts at today) src/ppt/RedemptionManager.sol#L910 (getOverdueLiability - static variable, no rollover logic found)

Solution

It’s recommended to implement a rollover mechanism to add the overdueLiability.

Status

Fixed

[N16] [Medium] Unsafe Liability Deduction in _removeLiability

Category: Design Logic Audit

Content

The _removeLiability function indiscriminately deducts from overdueLiability if settlementTime < today, assuming the liability was previously rolled over. When settleRedemption or settleWithVoucher is called for a request settled in the past (e.g., Request due on Day 5, settled on Day 6), it triggers _removeLiability. _removeLiability checks if (dayIndex < today). If true, it blindly subtracts the amount from overdueLiability.

Due to the missing rollover mechanism (N15), the specific liability for Day 5 may never have been added to overdueLiability. If overdueLiability currently holds legitimate old debt (e.g., 100k from Day 1), settling the Day 5 request (50k) will subtract 50k from the Day 1 record.

This completely corrupts the internal accounting. The system effectively "pays off" Day 1's recorded debt using Day 5's settlement action. Permanent loss of debt records: The 50k debt record for Day 1 is erased. This makes overdueLiability artificially lowered, increases the available quota, and allows further drain of the system.

Code location:

src/ppt/RedemptionManager.sol#L891-L894

```
    function _removeLiability(uint256 settlementTime, uint256 amount) internal {
        ...
        if (dayIndex < today) {
            uint256 toRemoveFromOverdue = overdueLiability >= amount ? amount :
overdueLiability;
            overdueLiability -= toRemoveFromOverdue;
        }
        ...
    }
```

Solution

The system needs a way to track whether a specific day's liability has been rolled over. Alternatively, ensure the "Rollover" mechanism is strictly atomic and always up-to-date before any settlement can occur (e.g., check and trigger rollover at the start of _removeLiability).

Status

Fixed

[N17] [Information] Indefinite Fund Lock via Pending Approval

Category: Design Logic Audit

Content

The cancelRedemption function in RedemptionManager is permanently disabled (revert CancellationDisabled()). If a user submits a redemption that requires approval (e.g., > 50k USDT), the assets are moved to the Vault as "Pending Approval Shares". If the KEEPER_ROLE is inactive, malicious, or simply negligent, the user's funds are locked indefinitely. The user cannot withdraw the request to regain their shares.

Code location:

src/ppt/RedemptionManager.sol#L343-L346

```
    function cancelRedemption(uint256 requestId) external override nonReentrant {
        // Redemption cancellation is disabled
        revert CancellationDisabled();
    }
```

Solution

It's recommended to enable cancelRedemption for requests that are in the PENDING_APPROVAL state.

Status

Acknowledged; After communicating with the project team, they stated that this was the intended design because RWA asset settlement has a fixed cycle. This design is to prevent users from intentionally redeeming their assets due to price increases during RWA settlement.

[N18] [Low] Missing Role Assignment for Liquidation

Category: Authority Control Vulnerability Audit

Content

The RedemptionManager._executeSettlement function calls assetController.executeWaterfallLiquidation when available cash is insufficient. AssetController.executeWaterfallLiquidation is restricted by onlyRole(REBALANCER_ROLE). But there is no code in RedemptionManager.initialize or PPT.initialize that grants the RedemptionManager contract the REBALANCER_ROLE on AssetController. If not manually configured, this may lead to denial of service for redemptions during low liquidity.

Code location:

src/ppt/RedemptionManager.sol#L706-L717

src/ppt/AssetController.sol#L446

```
            if (address(assetController) != address(0)) {
                uint256 funded = assetController.executeWaterfallLiquidation(
                    deficit,
                    PPTTypes.LiquidityTier.TIER_1_CASH  // Only liquidate L1 yield
assets
                );
```

```
        ...
    }
```

Solution

It's recommended to ensure the deployment and initialization script explicitly grants REBALANCER_ROLE to the RedemptionManager address on the AssetController contract.

Status

Acknowledged

[N19] [Low] Inflexible Voucher Threshold Configuration

Category: Design Logic Audit

Content

The logic in _approveRedemptionCore handles the voucher threshold with a hardcoded fallback. If the admin sets voucherThreshold to 0 (intending for immediate vouchers), the code forces it to 7 days. This prevents setting a strategy of "always mint NFT vouchers immediately".

Code location:

src/ppt/RedemptionManager.sol#L480

```
uint256 threshold = voucherThreshold > 0 ? voucherThreshold : 7 days;
```

Solution

If 0 is a valid value, then the condition >= 0 is used.

Status

Fixed

[N20] [Low] Misleading Settlement Estimate in Previews

Category: Design Logic Audit

Content

The previewRedemption and previewEmergencyRedemption functions return an estimatedSettlementTime that assumes the request will be approved immediately (e.g., block.timestamp + settlementDelay). However, if requiresApproval is true, the actual request logic (requestRedemption / requestEmergencyRedemption) sets

settlementTime to 0 (indicating "pending approval"). The real settlement time will be ApprovalTime + settlementDelay. This discrepancy can mislead front-ends or users into believing funds will arrive at a specific time (e.g., T+7 or T+0), whereas the actual time includes an indefinite administrative wait period.

Code location:

src/ppt/RedemptionManager.sol#L379, L406

```
    function previewRedemption(uint256 shares) external view override returns
(PPTTypes.RedemptionPreview memory preview) {
        ...
        preview.estimatedSettlementTime = block.timestamp +
PPTTypes.STANDARD_REDEMPTION_DELAY;
        ...
    }

    function previewEmergencyRedemption(uint256 shares) external view override
returns (PPTTypes.RedemptionPreview memory preview) {
        ...
        preview.estimatedSettlementTime = block.timestamp +
PPTTypes.EMERGENCY_REDEMPTION_DELAY;
        ...
    }
```

Solution

It's recommended to modify both preview functions to return 0 for estimatedSettlementTime when requiresApproval is true, or clarify in the documentation/interface that this value is conditional on immediate approval.

Status

Fixed

[N21] [Medium] Data Inconsistency in Liability Adjustments

Category: Design Logic Audit

Content

The administrative functions adjustOverdueLiability and adjustDailyLiability allow the KEEPER_ROLE to manually modify the internal liability tracking variables of the RedemptionManager. However, these functions do not make corresponding calls to the Vault (PPT) contract to update its totalRedemptionLiability. This creates a permanent desynchronization between the Manager's detailed records and the Vault's global accounting. Since the Vault uses

totalRedemptionLiability to calculate totalAssets and sharePrice, this inconsistency can lead to incorrect Share Price (NAV) calculations.

Code location:

src/ppt/RedemptionManager.sol#L937-L946

```
    function adjustOverdueLiability(uint256 amount) external onlyRole(KEEPER_ROLE) {
        overdueLiability = amount;
        emit AdjustOverdueLiability(amount);
    }

    function adjustDailyLiability(uint256 dayIndex, uint256 amount) external
onlyRole(KEEPER_ROLE) {
        dailyLiability[dayIndex] = amount;
         emit AdjustDailyLiability(dayIndex, amount);
    }
```

Solution

It's recommended to modify these adjustment functions to also call the corresponding addRedemptionLiability or removeRedemptionLiability functions on the Vault contract to ensure that the global liability state remains synchronized with the local state.

Status

Fixed

[N22] [Low] Misleading Voucher Face Value due to Fee Discrepancy

Category: Design Logic Audit

Content

In the approveRedemptionWithDate function, the netAmount encoded into the NFT Voucher is calculated using the current global fee configuration (baseRedemptionFeeBps or emergencyPenaltyFeeBps) at the moment of approval. However, during the actual settlement in _executeSettlement, the payout amount is calculated using the estimatedFee which was snapshot and stored in the request struct at the time of the application.

If the admin updates the fee configuration between the request time and the approval time, the face value stored in the NFT Voucher will mismatch the actual redeemable amount. This creates a situation where a voucher might display a value of 1000 USDT (calculated with a new, lower fee) but only pay out 990 USDT (calculated with the old,

higher fee) when redeemed. This discrepancy allows for potential fraud in the secondary market.

Code location:

src/ppt/RedemptionManager.sol#L482-L498

```
        if (delay > threshold && address(redemptionVoucher) != address(0)) {
            // Calculate net amount (netAmount = grossAmount - fee)
            bool isEmergency = request.channel ==
PPTTypes.RedemptionChannel.EMERGENCY;
            uint256 fee = _calculateRedemptionFee(request.grossAmount, isEmergency);
            uint256 netAmount = request.grossAmount - fee;

            // Generate NFT voucher (using net amount)
            uint256 tokenId = redemptionVoucher.mint(
                request.owner,
                requestId,
                netAmount,  // Net amount, not grossAmount
                settlementTime
            );
            request.hasVoucher = true;

            emit VoucherMinted(requestId, tokenId, request.owner);
        }
```

Solution

It's recommended to ensure consistency between the voucher's face value and the actual settlement payout. Either use the stored request.estimatedFee when calculating the voucher's netAmount, or update the request.estimatedFee to the current fee value during the approval process before minting the voucher.

Status

Fixed

[N23] [Medium] Unbounded Settlement Delay by Keeper

Category: Design Logic Audit

Content

The approveRedemptionWithDate function allows the KEEPER_ROLE to specify a customSettlementTime for a redemption request. While the function enforces a minimum delay check (customSettlementTime >= block.timestamp + minDelay), it does not enforce any maximum limit. A compromised or malicious Keeper could approve a redemption with a settlement time set to the distant future. Since the cancelRedemption function is permanently

disabled, the user's funds would be effectively locked indefinitely with no recourse.

Code location:

src/ppt/RedemptionManager.sol#L464-L473

```
        if (customSettlementTime == 0) {
            // Use default delay
            settlementTime = block.timestamp + minDelay;
        } else {
            // Validate custom time >= minimum delay
            if (customSettlementTime < block.timestamp + minDelay) {
                revert InvalidSettlementTime(customSettlementTime, block.timestamp +
minDelay);
            }
            settlementTime = customSettlementTime;
        }

        request.settlementTime = settlementTime;
```

Solution

It's recommended to implement a maximum settlement delay check (e.g., require(customSettlementTime <= block.timestamp + MAX_DELAY, "Delay too long")) to prevent indefinite locking of user funds. MAX_DELAY should be a reasonable constant (e.g., 30 days).

Status

Fixed

[N24] [High] Risk of Excessive Privilege

Category: Authority Control Vulnerability Audit

Content

The protocol concentrates significant power in privileged roles (DEFAULT_ADMIN_ROLE, ADMIN_ROLE, KEEPER_ROLE, OPERATOR_ROLE) across core contracts. These roles can arbitrarily manipulate user funds, asset valuation, and redemption processes, creating a single point of failure.

1.Root of Trust: The DEFAULT_ADMIN_ROLE (inherent in OpenZeppelin's AccessControl) is the ultimate superuser. It can grant and revoke any role, including ADMIN_ROLE, OPERATOR_ROLE, and UPGRADER_ROLE. If this role is compromised, the attacker gains full control over the entire system, bypassing any separation of duties.

2.Universal Upgradeability Risk All core contracts (AssetController, PPT, RedemptionManager, RedemptionVoucher) are UUPS upgradeable. The UPGRADER_ROLE (typically held by DEFAULT_ADMIN_ROLE or a Timelock) can call _authorizeUpgrade to replace the contract implementation.

Code location:

All onlyRole protected functions in the listed contracts.

```
AssetController Contract
Admin/Upgrader:
_authorizeUpgrade: Replace contract logic.
addAsset / removeAsset: Manipulate the asset whitelist, potentially burning asset
value from NAV.
updateAssetConfig / setLayerConfig: Change critical valuation parameters and liquidity
tiers.
setAssetActive: Enable/disable assets, allowing "Flash Value Injection".
setOracleAdapter / setSwapHelper: Replace price feeds and swap logic with malicious
contracts.
setDelayedSettlementAsset / setDefaultSettlementTimeout: Manipulate delayed settlement
rules.
Rebalancer/Operator:
purchaseAsset / redeemAsset: Trigger asset trades.
executeWaterfallLiquidation: Trigger massive forced sell-offs.
purchaseDelayedAsset / redeemDelayedAsset: Create unverified pending settlements (NAV
inflation).
confirmSettlement: Confirm settlements without proof.

RedemptionManager Contract
Admin/Upgrader:
_authorizeUpgrade: Replace contract logic.
setAssetController: Replace the entire asset management module.
setVoucherThreshold: Change NFT generation rules.
setStandardApprovalAmount / setStandardApprovalQuotaRatio: Manipulate T+7 channel
thresholds.
setEmergencyApprovalAmount / setEmergencyApprovalQuotaRatio: Manipulate T+0 channel
thresholds.
Keeper:
approveRedemption / approveRedemptionWithDate: Arbitrarily approve or delay
redemptions (indefinite lock).
rejectRedemption: Arbitrarily reject requests.
adjustOverdueLiability / adjustDailyLiability: Manually tamper with liability records.

PPT Contract (Vault)
Admin/Upgrader:
_authorizeUpgrade: Replace contract logic.
setAssetController / setRedemptionManager: Re-link core modules.
setStandardQuotaRatio / refreshEmergencyQuota: Manipulate quota limits.
```

```
Operator:
lockShares / unlockShares / burnLockedShares: Arbitrarily manipulate share states.
addRedemptionLiability / removeRedemptionLiability: Arbitrarily manipulate liability
(NAV impact).
addRedemptionFee / reduceRedemptionFee: Arbitrarily manipulate fee revenue.
transferAssetTo / approveAsset: Directly drain funds.
reduceEmergencyQuota / restoreEmergencyQuota: Manipulate emergency quotas.
addPendingApprovalShares / removePendingApprovalShares: Freeze user shares.
convertPendingToLocked: Force lock user shares.
resetLockedMintAssets: Reset minting locks.

RedemptionVoucher Contract
Admin/Upgrader:
_authorizeUpgrade: Replace contract logic.
Minter:
mint: Create fake debt obligations.
burn: Destroy user vouchers.
```

Solution

In the short term, transferring owner ownership to multisig contracts is an effective solution to avoid single-point risk. But in the long run, it is a more reasonable solution to implement a privilege separation strategy and set up multiple privileged roles to manage each privileged function separately. And the authority involving user funds should be managed by the community, and the EOA address can manage the authority involving emergency contract suspension. This ensures both a quick response to threats and the safety of user funds.

Status

Acknowledged; The contract has been deployed on BSC mainnet with each role assigned to its designated address. Specifically, both DEFAULT_ADMIN_ROLE and ADMIN_ROLE are governed by a 4-of-5 multisig contract, while UPGRADER_ROLE is controlled by a Timelock contract. KEEPER_ROLE and REBALANCER_ROLE are assigned to two EOA addresses. Given this separation of privileges, the overall risk rating of the contract has been downgraded from High to Medium at the Audit Result.

[N25] [Information] Fixed-Liability Redemption Model Creates "Bank Run" Incentive

Category: Others

Content

The PPT system uses a "Book Value" model where a redemption request locks the NAV at the moment of request (converting shares to a fixed USDT liability). effectiveSupply is reduced, but totalAssets is reduced by the fixed USD

liability amount. This concentrates volatility risk on the remaining holders. For example: Vault has 100 USDT assets and 100 Shares (NAV = 1.0). User A redeems 50 Shares (Liability = 50 USDT). Remaining assets = 50 USDT, Remaining Shares = 50. If assets drop 10% (to 90 Total), Liability is still fixed at 50 USDT. New Net Assets = 40 USDT (90 - 50). New NAV = 40 / 50 = 0.8. The redeeming user got out at 1.0, while remaining users took a 20% hit (leveraged loss). This creates a race condition during market downturns.

System Essence: The system operates on an "On-Demand Liquidity" model, NOT a "Full Redemption Guarantee". Users are not guaranteed to exit at any time; they are competing for finite liquidity associated with a specific settlement date.

Risk Nature: Unfairness & Leveraged Loss: The risk is not merely about illiquidity, but fundamentally about fairness.

First-Mover Advantage: Users who redeem early lock in today's NAV (e.g., 1.0).

Leveraged Loss: If assets depreciate after the liability is booked (but before settlement), the fixed liability (e.g., 50 USDT) remains constant. The entire loss falls on the remaining asset pool, causing a leveraged drop in NAV for remaining shareholders.

Example:

Vault 100 Assets / 100 Shares. User A redeems 50 shares at 1.0 NAV. Liability = 50.

Assets drop 10% to 90.

Net Assets = 90 - 50 (Liability) = 40.

Remaining Shares = 50.

New NAV = 0.8 (20% loss for remaining users, while User A exited at par).

Liquidation Scope (Tier 1 Constraint): The "Bank Run" incentive is exacerbated by the Tier 1 Liquidation Restriction.

When liquidity is insufficient, RedemptionManager calls executeWaterfallLiquidation.

However, it is hardcoded to liquidate only TIER_1_CASH assets.

If the vault holds significant value in Tier 2/3 assets (RWAs), these cannot be automatically liquidated to satisfy redemptions.

This creates a scenario where the "door closes" as soon as Tier 1 liquidity is exhausted, trapping users even if the protocol technically has backend assets.

Code location:

src/ppt/PPT.sol#L172-L182

src/ppt/RedemptionManager.sol#L546-L605

```
function totalAssets() public view override returns (uint256) {
    uint256 grossValue = _getGrossAssets();

    // Deduct redemption liability
    if (totalRedemptionLiability >= grossValue) return 0;
    uint256 netValue = grossValue - totalRedemptionLiability;

    // Deduct withdrawable fees
    if (withdrawableRedemptionFees >= netValue) return 0;
    return netValue - withdrawableRedemptionFees;
}

function _processStandardRedemption(
    address owner,
    uint256 shares,
    address receiver,
    uint256 grossAmount,
    uint256 nav
) internal returns (uint256 requestId) {
    ...
}
```

Solution

N/A

Status

Acknowledged

[N26] [Information] Token Compatibility Reminder

Category: Others

Content

This protocol's accounting logic (NAV calculation, share pricing, settlement) relies on standard ERC20 behavior. It is fundamentally incompatible with weird ERC20 tokens, specifically:

1.Fee-on-Transfer Tokens (Deflationary) : The protocol tracks balances based on transfer amounts. If a token deducts a fee during transfer, the Vault will receive less than expected, but the accounting system (e.g., totalAssets) might record the pre-fee amount. This creates a permanent discrepancy between "book value" and "actual balance,"

leading to insolvency.

2.Rebasing Tokens (Inflationary/Elastic) : The protocol assumes that balanceOf(vault) only changes when transfer or transferFrom is called. If a token's balance changes automatically (rebase), the Vault's internal accounting (e.g., cash , layer1Liquidity) will not reflect this change until a manual sync occurs (if any). This can lead to incorrect Share Price calculations (either overvalued or undervalued).

3.Callback/Hook Tokens (ERC777/ERC677) : Tokens that trigger callbacks (like tokensReceived) can introduce reentrancy vectors that the current nonReentrant modifiers might not fully cover, especially in complex cross-contract interactions like AssetController liquidations.

Solution

Strictly whitelist only standard, fixed-balance ERC20 tokens (like USDC, USDT standard implementations) as underlying assets or purchased assets. Do not support deflationary, inflationary, or callback-enabled tokens.

Status

Acknowledged

[N27] [Low] Potential DoS in AssetController via Unbounded Asset Loop

Category: Denial of Service Vulnerability

Content

The calculateAssetValue function in AssetController iterates through all configured assets to calculate the total value. There is no hard limit on the number of assets that can be added to the _assetConfigs array. If too many assets are added, or if specific asset balanceOf / decimals calls are expensive (or malicious), this loop could exceed the block gas limit. Since deposit and mint functions in the PPT contract force a call to calculateAssetValueFresh() (which executes this loop via _calculateAssetValueInternal), this would freeze all deposits and redemptions effectively causing a Denial of Service.

Code location:

src/ppt/AssetController.sol#L622-L649

```
    function _calculateAssetValueInternal() internal view returns (uint256 totalValue)
{
        ...
        for (uint256 i = 0; i < _assetConfigs.length; i++) {
            ...
```

```
            uint256 balance = IERC20(config.tokenAddress).balanceOf(address(vault));
            if (balance == 0) continue;
            ...
        }
        ...
    }
```

Solution

It's recommended to implement a strict limit on the maximum number of active assets that can be added (e.g., MAX_ASSETS = 20).

Status

Acknowledged

[N28] [Low] NAV Manipulation via Unchecked expectedUsdt Input

Category: Design Logic Audit

Content

The redeemDelayedAsset and purchaseDelayedAsset functions in AssetController accept a expectedUsdt (or expectedTokenValue) argument provided by the caller (REBALANCER_ROLE). This value is directly added to totalPendingValue, which is a component of the vault's totalAssets() calculation. There is no validation that this expectedUsdt value corresponds to the actual market value of the assets being settled. A compromised or malicious Rebalancer can provide an arbitrarily large value, artificially inflating the Vault's NAV. This allows for massive share price manipulation, potentially facilitating theft of funds via redemption of pre-held shares at inflated prices. Even if the Rebalancer is trusted, a simple typo (human error) could critically destabilize the system's pricing.

Code location:

src/ppt/AssetController.sol#L997-L1036

```
    function redeemDelayedAsset(
        address token,
        uint256 tokenAmount,
        uint256 expectedUsdt
    ) external override onlyRole(REBALANCER_ROLE) nonReentrant whenNotPaused returns
(uint256 settlementId) {
        ...
        pendingSettlements[settlementId] = PendingSettlement({
            ...
            expectedUsdtValue: expectedUsdt,
```

```
            ...
        });
        ...
        totalPendingValue += expectedUsdt;

        emit SettlementCreated(settlementId, PPTTypes.SettlementType.SALE, token,
tokenAmount, expectedUsdt);
        _invalidateCache();
    }
```

Solution

It's recommended to remove the expectedUsdt argument. The contract should calculate the expected value internally using the canonical price from oracleAdapter: expectedUsdt = oracleAdapter.getPrice(token) * tokenAmount. This ensures totalPendingValue reflects the true market value of the pending settlement.

Status

Acknowledged; In the fix version, the `redeemDelayedAsset` function was removed, but it still existed in the `purchaseDelayedAsset` function. Furthermore, `purchaseDelayedAsset` was changed to be controlled by `DELAYED_ADAPTER_ROLE`.

[N29] [Medium] Griefing and Settlement Breaking via Public checkAndMarkAbnormal

Category: Design Logic Audit

Content

The checkAndMarkAbnormal function allows anyone to mark a pending settlement as ABNORMAL once the defaultSettlementTimeout (default 7 days) has passed. Once a settlement is marked ABNORMAL:

The value is subtracted from totalPendingValue (NAV drops). And the status changes to ABNORMAL. Crucially, the confirmSettlement function reverts if the status is not PENDING.

Mechanism: If a legitimate off-chain settlement is slightly delayed (e.g., arrives on Day 7 + 1 second), a malicious actor can front-run the Keeper's confirmSettlement call by calling checkAndMarkAbnormal first. This transitions the settlement to ABNORMAL. When the Keeper's transaction attempts to execute, it fails because status != PENDING. The valid settlement logic is permanently blocked as a Settlement Deadlock. The system cannot recognize that the funds have arrived. The NAV artificially drops because totalPendingValue is reduced, even though the assets might be safely in the vault. And there is no function in the contract to revert an ABNORMAL status back to PENDING or to force-settle an ABNORMAL record.

Code location:

src/ppt/AssetController.solL1101-L1118

```
    function checkAndMarkAbnormal(uint256 settlementId) external override {
        ...
        if (block.timestamp <= settlement.deadline) {
            revert SettlementNotTimedOut(settlementId, settlement.deadline,
block.timestamp);
        }
        totalPendingValue -= settlement.expectedUsdtValue;
        settlement.status = PPTTypes.SettlementStatus.ABNORMAL;
        ...
    }
```

Solution

It's recommended to restrict checkAndMarkAbnormal to KEEPER_ROLE or ADMIN_ROLE only.

Status

Fixed

[N30] [Suggestion] Missing parameters check

Category: Others

Content

In the following function call and construct, invoking the contract can modify or set the address, but there is no check for the zero address.

In the following function calls and constructors, invoking the contract can modify or set the parameters, but there is no validation check for the parameters (e.g., zero address checks or boundary limits).

1.Missing Zero Address Checks The following functions allow setting critical contract addresses or parameters to address(0) without validation, which could break protocol functionality or cause DoS.

- AssetController :
  - setOracleAdapter(address)
  - setSwapHelper(address)
- RedemptionManager :
  - initialize : The redemptionVoucher

    - setAssetController(address)
- PPT :
    - setAssetController(address)
    - setRedemptionManager(address)

2.Missing Parameter Limit Checks The following functions allow setting numerical parameters to extreme values (e.g., 0 or MAX_UINT256) without sanity checks, potentially leading to economic exploits or broken logic.

- RedemptionManager :
    - setStandardApprovalAmount : No max limit. Could be set to 0 (forcing all requests to require approval) or MAX (bypassing approval entirely).
    - setEmergencyApprovalAmount : No max limit.
    - setVoucherThreshold : No max limit. Could be set to 100 years, effectively disabling voucher generation.
- AssetController :
    - setDefaultSettlementTimeout : No limits. Could be set to 0 (immediate timeout) or MAX (never timeout).

Code location:

src/ppt/AssetController.sol#L936,L944, L985

src/ppt/RedemptionManager.sol#L195, L964, L900,L1007,L956

src/ppt/PPT.sol#L501,L536

Solution

It is recommended to perform appropriate range checks or 0 value checks on the corresponding parameters.

Status

Acknowledged

[N31] [Suggestion] Missing the event records

Category: Others

Content

The following functions are missing event records after calling the modification.

Code location:

src/ppt/AssetController.sol#L443

src/ppt/PPT.sol#L426-L432, L459

```
function executeWaterfallLiquidation
function addRedemptionLiability
function removeRedemptionLiability
function reduceEmergencyQuota
```

Solution

It is recommended to record events when sensitive parameters are modified for self-inspection or community review.

Status

Fixed

[N32] [Information] External Call Reminder

Category: Unsafe External Call Audit

Content

The system logic heavily relies on the correctness and security of external contracts which are out of scope for this audit.

OracleAdapter : Critical for asset pricing and NAV calculation. Malfunction leads to wrong share price.

SwapHelper : Used for asset liquidation and purchasing. Malfunction leads to fund loss errors.

PurchaseAdapter : Used for OTC asset interactions.

If any of these external components are malicious, upgradeable to malicious versions, or contain bugs, the PPT and AssetController contracts can be drained or compromised.

Solution

N/A

Status

Acknowledged

[N33] [Low] Broken Slippage Protection in Purchase Logic

Category: Design Logic Audit

Content

The _calculateMinOutput function in AssetController (Fix Version) assumes that the input token (tokenIn) for a purchase is always pegged 1:1 to USD (like USDT). It fails to query the Oracle for the price of tokenIn. The function calculates expected output as `uint256 expectedOutput = (amountIn18 * (10 ** decimals)) / price;` This formula is only correct if tokenIn is a stablecoin worth $1. If the input token is not a stablecoin (e.g., WBTC, ETH), the function drastically undervalues the input.

Code Location:

src/ppt/AssetController.sol#L697-728

```
function _calculateMinOutput(
    address tokenOut,
    uint256 amountIn,
    uint256 maxSlippageBps
) internal view returns (uint256 minOutput) {
    ...
    uint256 expectedOutput = (amountIn18 * (10 ** decimals)) / price;
    ...
}
```

Solution

It's recommended to update _calculateMinOutput to fetch the Oracle price of tokenIn and use it to calculate the USD value of the input amount.

Status

Fixed

[N34] [Low] Inconsistent Slippage Configuration in Asset Sale

Category: Design Logic Audit

Content

The _sellAsset function exhibits inconsistent behavior regarding slippage configuration. The OTC path correctly respects the asset-specific maxSlippage config, while the SWAP path ignores it and forces the use of the global defaultSwapSlippage. If an asset requires higher slippage (e.g., 3%) but the global default is strict (1%), sales via Swap/DEX will revert, potentially blocking rebalancing or redemptions.

MEV Exposure: If an asset needs tight slippage (0.1%) but the global default is loose (1%), trades are exposed to unnecessary front-running/sandwich attacks.

Code Location:

src/ppt/AssetController.sol#L979-980, L991

```
        try swapHelper.sellRWAAsset(
             token, vaultAsset, tokenAmount, defaultSwapSlippage, address(vault)
         ) returns (uint256 reported) {
             ...
             if (address(oracleAdapter) != address(0)) {
                 uint256 minOutput = _calculateMinOutputForSell(token, tokenAmount,
defaultSwapSlippage);
                 if (actualReceived < minOutput) {
                     revert OutputBelowMinimum(actualReceived, minOutput);
                 }
             }
```

Solution

It's recommended to update the SWAP path to use the same logic as the OTC path.

Status

Fixed

[N35] [Information] Logic Order Error leads to Locked Funds Failure

Category: Design Logic Audit

Content

The purchaseAsset function resets the lockedMintAssets counter to zero before calling _executePurchase. However, _executePurchase relies on reading lockedMintAssets to ensure it doesn't spend funds that should be locked (e.g., for cooling-off periods or pending redemptions). purchaseAsset calls vault.resetLockedMintAssets(). This sets the Vault's lockedMintAssets state variable to 0.Then purchaseAsset calls _executePurchase. The _executePurchase reads uint256 lockmint = vault.lockedMintAssets();. This value is now guaranteed to be 0. The availability check availableBalance = vaultBalance - (withdrawable + lockmint) becomes ineffective regarding the lock component. The Rebalancer can use funds that were supposed to be locked to purchase risky assets. This violates the safety guarantees of the "Locked Mint" mechanism (if used to secure minting cooldowns or redemption reservations).

Code Location:

src/ppt/AssetController.sol#L429, L822

```
    function purchaseAsset(
        address tokenIn,
        address tokenOut,
        uint256 amountIn
    ) external override onlyRole(REBALANCER_ROLE) nonReentrant whenNotPaused returns
(uint256 tokensReceived) {
        ...
        vault.resetLockedMintAssets();
        (uint256 spent, uint256 received) = _executePurchase(index - 1,
actualTokenIn, amountIn);
        ...
    }

       if (tokenIn == vaultAsset) {
            ...
            uint256 lockmint = vault.lockedMintAssets();
            uint256 totalDeductions = withdrawable + lockmint;
            ...
        } else {
            ...
        }
```

Solution

It is recommended to clearly define the design logic and call resetLockedMintAssets after the _executePurchase and at the end of the purchaseAsset function.

Status

Acknowledged

[N36] [Information] Rollover Logic Gas Limit Risk (DoS)

Category: Denial of Service Vulnerability

Content

The `_rolloverOverdueLiabilities` function iterates from `lastRolloverDay` to the current day (`today`). If the contract remains inactive (no liability-changing transactions) for a very long period (e.g., several years), the loop iterations can grow large enough to consume all available gas, exceeding the block gas limit. The contract becomes permanently "bricked" for standard operations. Users cannot request redemptions, and the admin cannot perform

normal maintenance if it triggers the same rollover logic.

Code Location:

src/ppt/RedemptionManager.sol#L910-916

```
function _rolloverOverdueLiabilities() internal {
    ...
    for (uint256 day = lastRolloverDay; day < today; day++) {
        uint256 amount = dailyLiability[day];
        if (amount > 0) {
            overdueLiability += amount;
            emit LiabilityRolledOver(day, amount);
        }
    }
    ...
}
```

Solution

N/A

Status

Acknowledged

[N37] [High] Share Price Manipulation

Category: Design Logic Audit

Content

The PPT vault uses effectiveSupply() (calculated as totalSupply() - totalLockedShares()) as the denominator in its share pricing logic (_convertToShares and _convertToAssets). This design is intended to exclude redeeming shares from the valuation of the active pool. However, this creates a critical edge case: if an attacker (or a group of users) redeems all outstanding shares, effectiveSupply drops to zero while totalSupply remains non-zero. In this state, the standard ERC4626 pricing formula breaks down. Specifically, if the vault still holds assets (e.g., via direct donation or residual balances), the formula shares = assets * (supply + 1) / (totalAssets + 1) will calculate 0 shares for new deposits whenever supply is 0 and totalAssets is sufficiently large. A legitimate user depositing funds into a "seemingly empty" (zero effective supply) vault could lose 100% of their deposit if the vault has any residual assets, as they would receive 0 shares in return.

Scenario:

Initial State: Attacker deposits 10e18 USD (assuming 1:1 rate) to mint 10e18 PPT shares (minimum required).

totalSupply = 10e18, effectiveSupply = 10e18. totalAssets = 10e18 USD.

Locking: Attacker requests redemption for all 10e18 shares.

lockedShares = 10e18.

effectiveSupply = 10e18 - 10e18 = 0.

Donation (or Residual): Attacker donates 1000 USD to the vault.

totalAssets = 1010 USD.

Victim Deposit: Victim deposits 500 USD, expecting to become a shareholder.

Input: assets = 500.

Calculation: 500 * (0 + 1) / (1010 + 1).

Result: 500 / 1011 = 0 Shares.

Impact: The victim transfers 500 USD to the vault but receives 0 PPT shares. The victim's funds are now permanently trapped in the vault (or claimable by the attacker if they cancel their redemption), resulting in a 100% loss for the victim.

Code Location:

src/ppt/PPT.sol#L196 (sharePrice), L218 (_convertToShares).

```
    function _convertToShares(uint256 assets, Math.Rounding rounding) internal view
virtual override returns (uint256) {
        uint256 supply = effectiveSupply();
         uint256 totalAsset = totalAssets();
        if(supply>0){
            require(totalAsset>0, "totalAsset is not 0");
        }
        return assets.mulDiv(supply + 10 ** _decimalsOffset(), totalAsset + 1,
rounding);
    }
```

Solution

It’s recommended to override _decimalsOffset() in PPT to return like 18. AND ensure effectiveSupply cannot be manipulated to 0 if assets > 0.

Status

Fixed

[N38] [High] NAV Crash via Liability Mismanagement

Category: Design Logic Audit

Content

The `totalAssets` function subtracts `totalRedemptionLiability` from gross assets. If liabilities exceed gross assets, `totalAssets` returns 0, causing `sharePrice` to drop to 0 (or default). While `getStandardChannelQuota` prevents *automatic* requests from exceeding limits, the `approveRedemption` function (used by Keepers to process large/pending requests) lacks any solvency check.

1.Large requests exceed Quota and become `PENDING_APPROVAL`. Liability is NOT yet added.

2.Keeper calls `approveRedemption`. This function unconditionally calls `vault.addRedemptionLiability(grossAmount)`.

3.If the added liability (plus existing) exceeds `grossAssets` (e.g., due to asset depreciation since request time), `PPT.totalAssets()` reverts to 0.

4.Share price becomes 0. All subsequent redemptions yield 0 assets.

Catastrophic denial of service. A well-intentioned Keeper approval during a market downturn can inadvertently brick the fund's pricing mechanism.

Code Location：

src/ppt/PPT.sol# L217 (`totalAssets` return 0).

src/ppt/RedemptionManager.sol#L534 (`approveRedemption` adds liability without check).

Solution

It's recommended to add a strict check in `RedemptionManager.approveRedemption` to prevent liability from exceeding assets. Alternatively, it also recommend to add an insolvency check to _executeSettlement, or a global circuit breaker mechanism.

Status

Fixed

[N39] [Information] Rigid Historical Value Check in Settlement (DoS Risk)

Category: Denial of Service Vulnerability

Content

In the fixed version, the `confirmSettlement` function in `AssetController` validates the received asset value against the *historical* USDT value recorded at the time of purchase initiation.

This check (`_validateSettlementSlippage`) requires `CurrentValue >= OriginalValue * (1 - Slippage)`.

Scenario:

1.**T0**: Purchase 1 BTC at $50,000. `expectedUsdtValue` = 50,000.

2.**T+7**: Settlement arrives. Price drops to $48,000 (-4%).

3.**Validation**: `CurrentValue` ($48,000) < `OriginalValue` ($50,000) * 0.99 ($49,500).

4.**Result**: Transaction Reverts.

If asset prices drop during the settlement window (which can be days or weeks for RWA), the Rebalancer cannot confirm the settlement. The funds are stuck in the Vault, but the accounting remains in `PENDING` state, and the NAV remains artificially high (based on old $50k valuation).

This creates a Denial of Service (DoS) for settlements during high volatility.

Code Location:

src/ppt/AssetController.sol#Line 1365 (confirmSettlement); Line 1386 (_validateSettlementSlippage).

Solution

It's recommended to validate Amount instead of Value. If I bought 1 BTC, receiving 1 BTC is valid regardless of price.

Status

Acknowledged

5 Audit Result

Audit Number	Audit Team	Audit Date	Audit Result
0X002602020001	SlowMist Security Team	2026.01.26 - 2026.02.02	Medium Risk

Summary conclusion: The SlowMist security team uses a manual and the SlowMist team's analysis tool to audit the project. During the audit work, we found 9 high risks, 8 medium risks, 10 low risks, 3 suggestions, and 9 information. All the findings were acknowledged or fixed.

The contract has been deployed on BSC mainnet with each role assigned to its designated address. Specifically, both DEFAULT_ADMIN_ROLE and ADMIN_ROLE are governed by a 4-of-5 multisig contract, while UPGRADER_ROLE is controlled by a Timelock contract. KEEPER_ROLE and REBALANCER_ROLE are assigned to two EOA addresses. Given this separation of privileges, the overall risk rating of the contract has been downgraded from High to Medium.

6 Statement

SlowMist issues this report with reference to the facts that have occurred or existed before the issuance of this report, and only assumes corresponding responsibility based on these.

For the facts that occurred or existed after the issuance, SlowMist is not able to judge the security status of this project, and is not responsible for them. The security audit analysis and other contents of this report are based on the documents and materials provided to SlowMist by the information provider till the date of the insurance report (referred to as "provided information"). SlowMist assumes: The information provided is not missing, tampered with, deleted or concealed. If the information provided is missing, tampered with, deleted, concealed, or inconsistent with the actual situation, the SlowMist shall not be liable for any loss or adverse effect resulting therefrom. SlowMist only conducts the agreed security audit on the security situation of the project and issues this report. SlowMist is not responsible for the background and other conditions of the project.

Official Website

www.slowmist.com

E-mail

team@slowmist.com

Twitter

@SlowMist_Team

Github

https://github.com/slowmist